Lab 2 Part 3 — Calibration-Free Quantization of Qwen3-4B

Student ID: 314551011

Final result: WikiText-2 PPL **14.19** at a realized effective bit rate of **5.296 bits**.

Model	PPL
fp16 (same evaluation)	13.64
AdaptQuant (reference)	13.98
Ours	**14.19**

1. Method

1.1 Relation to AdaptQuant

- I did **not** reproduce AdaptQuant exactly. Its full text could not be retrieved while I was implementing this: OpenReview served an automated-browser check.
- I implemented my own method in the spirit of its title, *per-tile selection among hardware-aligned block formats*, and then took it further where my experiments showed a benefit.
- I cannot list precise differences from AdaptQuant's internals. The comparison below is limited to the design space that the title describes.

1.2 What is quantized

- The weights **and** the input activations of every `q/k/v/o/gate/up/down_proj` in the 36 decoder layers. Each `nn.Linear` is replaced by a `QLinear`.
- Not quantized: the embedding / tied `lm_head`, and the attention score / softmax path.
- Both tensors are **fake-quantized**: quantized to the target format and de-quantized to fp16. A T4 reproduces the numerics without Blackwell kernels.

1.3 Formats

Tensor	Element	Block scale	Other overhead	Bits / element
Weights, default tiles	FP4 E2M1	FP8 E4M3 per 32 weights	fp32 per tensor	4 + 8/32 = **4.25**
Weights, promoted tiles (4.5 %)	FP6 E2M3	FP8 E4M3 per 32	fp32 per tensor	6.25
Activations	FP6 E2M3	E8M0 (power of two) per 32 (OCP MXFP6)	—	**6.25**

- **Weight block scale:** s = amax(block) / 6, stored in E4M3 relative to the fp32 per-tensor scale g = amax(W) / (448 · 6). The block maximum therefore maps exactly onto the largest FP4 value and is never clipped.
- **Activation block scale:** computed **online** from each block as 2^(⌊log₂ amax⌋ – 2), following the OCP MX specification.

1.4 Per-tile format selection (weights)

1. **Tiles.** Each weight matrix is split into tiles of 16 output rows × 32 input columns, so a tile is 16 consecutive quantization blocks of one K-slice.
2. **Scoring.** Each tile is quantized as FP4 and as FP6. Its score is the reduction in squared reconstruction error, normalized by the mean square of its layer so scores are comparable across layers.
3. **Promotion.** The 4.5 % of tiles with the highest scores, taken globally over all 252 matrices, are stored as FP6. Every tile carries a 1-bit format tag.

1.5 Why it is calibration-free

- No calibration set is loaded and no model forward pass is used to make any decision.

- **Weights:** the format, scales and promoted tiles are determined from the weight tensor alone.
- **Activations:** the block scales are computed from the current block itself while the model runs.
- **No search or seed tuning:** there are no thresholds tuned per model, and no random seed. The method is fully deterministic.

1.6 Differences from the title-level design of AdaptQuant

- **Hardware alignment of the weight format.** The selection is between weight formats only, at a K-slice granularity that a tensor-core tile can consume. The weight path is not a native Blackwell format, however:
  - I use FP4 elements with an **E4M3 scale per 32**, which differs from both native formats: NVFP4 uses an E4M3 scale per 16, and MXFP4 a power-of-two scale per 32.
  - The reason is that E8M0 power-of-two scales **clip** a block's largest weights (Section 3.2). Making that clipping-free costs block 16 in NVFP4, which does not fit the 5.3-bit budget.
- **Activations stay uniform.** They use plain MXFP6. A dynamic per-tile FP4/FP6 choice for activations would save bits, but FP4 activations are very damaging (Section 3.1).

2. Result

	Weight bits	Activation bits	Effective bits	PPL
fp16	16	16	16	13.638
Final method	**4.342**	**6.250**	**5.296**	**14.194**

Weight bits:

- Promoted fraction f = 0.045 of 16×32 tiles.
- W = (1 – f)·4.25 + f·6.25 + 1/(16·32) = 4.25 + 0.090 + 0.002 = **4.342 bits/weight**.
- The fp32 per-tensor scales add 32 bits per matrix, below 10^{-5} bits/weight.

Activation bits: A = 6 + 8/32 = **6.25 bits/element**.

Effective bit rate: (W + A) / 2 = (4.342 + 6.25) / 2 = **5.296 ≤ 5.3**.

The rate is reproduced by `bash run.sh`, which recomputes the bit accounting in `aquant.py` while quantizing. The PPL comes from `evaluate.eval_ppl_wikitext2` (WikiText-2 test, 2048-token windows), unchanged from Part 2.

3. Observations and discussion

All numbers below were measured with the same evaluation; fp16 is 13.64. The full log is in `results/sweep.jsonl`.

3.1 Weights vs. activations

Setting	bits	PPL
activations MXFP8 only	A 8.25	13.99
activations MXFP6 only	A 6.25	13.79
activations MXFP4 only	A 4.25	15.89
weights MXFP6 only	W 6.25	13.86
weights MXFP4 only	W 4.25	15.22
W MXFP4 + A MXFP6	5.25	15.30
W MXFP6 + A MXFP4	5.25	16.27

- Both tensors are nearly lossless at 6 bits and degrade sharply at 4.
- Activations suffer more from FP4 than weights do. Activation outlier channels are larger, and their scales cannot be chosen offline.
- With the budget fixed at 10.6 bits for W + A, the right split is therefore **4-bit-class weights and 6-bit activations**.
- MXFP6 activations (13.79) are even slightly better than MXFP8 (13.99). That is within measurement noise (see 3.4): FP6 is already precise enough for activations.

3.2 Scale precision matters more than block size

- MXFP4's power-of-two scale with the OCP floor rule maps a block maximum anywhere into [4, 8). Values above 6 are **clipped**, and in LLM weight blocks those are exactly the important large weights.
- **Choosing floor or ceil per block** (data-free, lower squared error) improves weight-only PPL from 15.22 to 14.96.
- An **E4M3 block scale** (non-power-of-two) never clips: weight-only NVFP4 with block 16 gives PPL 13.64–14.08 at 4.5 bits.
- At equal bits (4.25), FP4 with an E4M3 scale per 32 beats MXFP4 in every combined setting that does not use rotation (rotation is discussed in Section 3.4).
- The fp32 second-level scale works better **per tensor** than per output row, in both comparisons I ran: NVFP4 weight-only 13.64 vs. 14.08, and the final configuration 14.19 vs. 14.50 (the per-row run promoted 4 % instead of 4.5 % of the tiles).

3.3 Per-tile mixed precision

- Spending the last ~0.09 bits on promoting 4.5 % of the tiles to FP6 is the most effective use of the remaining budget. The tiles with the largest FP4 error, i.e. the outlier-heavy ones, gain the most.
- Promoting to FP8 instead (half as many tiles) or using 128-row tiles was not better.
- With 1-row tiles the 1-bit tags cost 1/32 bit/weight, which forces a smaller promoted fraction.
- Keeping the top 0.02 % weights as exact sparse fp16 outliers (48 bits each, SpQR-style) did **not** help (14.50–14.72): FP4 with exact block-max scaling already represents the extreme weights well.

3.4 Difficulty: random rotations are a lottery on this model

- A randomized 32×32 block-Hadamard rotation (QuaRot-style, data-free, aligned to the MX blocks) gave the best single number: **13.71**, and even 13.60 with sparse outliers. **I did not use it**, because it is not robust.

Rotation	seed 0	seed 1	seed 2
shared per input size	13.71	16.78	15.22
one per layer	14.86	15.32	15.51
per layer, seed chosen by weight MSE (data-free)	15.88	15.81	14.21
plain Hadamard (no signs)	15.18		
activations only (weights fp16), shared	13.64	13.68	13.68
weights only (activations fp16), shared	13.62	16.70	15.19

- The variance comes entirely from the **weights**; the rotated activations are stable.
- Rotating only one layer group at a time (q/k/v, o, gate/up or down) with the bad seeds gives 14.37–15.49. The damage is spread over all layers, not one catastrophic matrix.
- Choosing the seed by weight MSE does not work: it is not a proxy for the end-to-end error.
- Picking seed 0 because it scores best on the *test set* would be test-set tuning, and there is no calibration-free way to find a good seed. I therefore chose the deterministic method.
- More generally, these runs show that at ~4.3 weight bits **different rounding realizations of the same format change PPL by ±1**. Differences of a few tenths between configurations should not be over-interpreted.

3.5 Other difficulties

- **Larger rotations hurt with block scaling.** Hadamard blocks of 128 and 512 gave 15.84 and 16.22, vs. 15.17 without rotation. They spread an outlier across many quantization blocks and raise all their scales.
- **Bit accounting.** Every overhead is counted: block scales, per-row or per-tensor scales, format tags and sparse indices.
- **Memory for the per-tile selection.** On a 16 GB T4, the global selection has to work one layer at a time: the scores stay on the CPU and the quantized copies are recomputed. Peak GPU memory is ~10.4 GB.

3.6 Why quality still degrades, and possible improvements

- The remaining +0.55 PPL is almost entirely weight error (activations alone cost +0.15). FP4 has only 15 values: even with exact block maxima, small weights in a block with one large weight are rounded

coarsely.

- **Improvements that stay calibration-free:**
  - Error feedback inside a weight matrix without data, e.g. a GPTQ-like update with an identity or weight-derived Hessian.
  - Rotations chosen by a deterministic, provably incoherence-improving criterion instead of random signs.
  - Dynamic FP4/FP6 *activation* tiles by crest factor, to free bits for more FP6 weight tiles.
- **Limitations:**
  - Fake quantization only.
  - The weight format, FP4 with an E4M3 scale per 32, is not a native Blackwell format.
  - The ranking of close configurations is within realization noise.